REVISED CAPSTONE PROJECT PROPOSAL, REV. 2

GetPrio: A Multi-Tenant QR-Based Digital Queue Management and Platform Operations System

Updated proposal aligned with the current application scope and implementation

Track Concentration: Web Applications Development

Primary Targeted SDG: SDG 9 - Industry, Innovation and Infrastructure

Supporting SDGs: SDG 8 and SDG 11

Prepared by: Roberto Carlo Abella

Adviser: Prof. Jovelyn Cuizon

Date: May 2026

Project Summary

Subject Area

Web application development for digital queueing, service operations, and SaaS-style platform administration.

Aim

To develop a tenant-aware queue platform with QR joining, live queue visibility, vendor controls, notifications, payment records, and platform oversight.

Core Problem

Manual queueing creates crowding, uncertainty, missed turns, and repetitive service-desk work.

Expected Deliverable

A working web prototype with customer/vendor frontend, platform dashboard, Express API, and PostgreSQL database.

Introduction

Many service establishments still depend on manual queue logs, paper numbers, verbal announcements, or simple first-come-first-served lists. These methods are familiar, but they create recurring problems: customers must stay near the service area, staff repeatedly answer questions about queue progress, and missed turns or disorganized ticket handling can disrupt service flow.

Digital queue systems help address these problems by letting users join, monitor, and receive updates about their queue status. However, many available systems are enterprise-oriented, costly, appointment-focused, or limited to a single organization. Smaller local establishments often need a practical web platform that can support multiple independent vendors while still keeping each vendor's data, queue settings, locations, and staff access separated.

GetPrio responds to this gap by providing a tenant-aware queue platform that supports vendor self-service and platform-level administration. The revised application now includes operational requirements that make the system closer to a deployable software-as-a-service prototype: subscription plans, queue join fees, PayMongo-ready payment records, OTP-secured public joins, notification delivery logs, configurable public board design, staff/counter attribution, and platform-wide monitoring.

Statement of the Problem

1. How can a web application reduce the need for customers to physically wait near service counters?
2. How can vendors manage queue flow, staff assignments, service counters, and customer tickets in a tenant-specific dashboard?
3. How can customers receive reliable queue visibility and near-turn alerts without requiring a complex mobile application?
4. How can a platform administrator monitor tenants, payments, subscriptions, and operational activity across multiple vendors?

5. How can queue records, ticket numbers, payment records, notifications, and public board settings be stored securely and consistently?

Objectives

General Objective: To develop GetPrio, a multi-tenant QR-based digital queue management and platform operations system that improves customer flow, vendor service efficiency, and platform-level manageability through a responsive web application.

Specific Objectives:

1. Implement vendor registration, tenant creation, login, and role-based access.
2. Allow customers to join queues through QR-based public links and OTP-verified online forms.
3. Generate tenant-specific daily ticket numbers, lookup codes, and queue positions using PostgreSQL-backed counters.
4. Provide vendor controls for walk-in ticket issuance, call-next, serve-current, skip-current, ticket history, and customer tracking.
5. Support branch/location records, store hours, service counters, staff membership, and counter assignments.
6. Provide live public queue monitoring using Server-Sent Events.
7. Record near-turn and called-ticket notifications through email or SMS provider integrations.
8. Support payment and subscription workflows through queue join payment records, billing checkout sessions, billing events, and configurable subscription plans.
9. Provide a platform operations dashboard for monitoring tenants, users, queue fees, subscriptions, join payments, and billing events.
10. Allow vendors to customize public board themes, including branding colors and uploaded board assets.

Scope and Limitations

Scope

- Customer-facing landing, account, queue join, ticket lookup, and public queue monitor pages.
- Vendor dashboard for queue operations, locations, staff, clients, history, reports, and settings.
- Platform administrator dashboard for operational and billing oversight.
- REST API services for auth, public queue access, vendor operations, billing, and platform administration.

Limitations

- The system is a capstone prototype and may require additional production hardening.
- SMS, email, payment, OAuth, CAPTCHA, and object storage depend on configured third-party provider accounts.
- The queue model focuses on first-in-first-out flow and does not yet include appointment scheduling or priority lanes.
- Offline operation is not included.
- The platform dashboard is for administration, not full customer support

- PostgreSQL schema for tenants, users, tickets, counters, OTPs, notifications, themes, fees, plans, subscriptions, payments, and billing events.

case management.

Target Users and Personas

Vendor Owner or Queue Manager

Operates a service desk for a clinic, salon, campus office, government desk, or small business counter. Needs ticket issuance, queue movement controls, staff access, counters, history, and fewer repeated queue-status inquiries.

Vendor Staff

Assists with daily queue processing and needs controlled access to queue actions, customer records, and history without full owner-level settings or billing permissions.

Customer

Joins a queue on-site through a QR code or remotely through a join page. Needs a simple form, a clear ticket number, queue visibility, and optional near-turn notifications.

Platform Administrator

Monitors the GetPrio platform through visibility into tenant activity, users, payment records, subscription plans, fee settings, and billing events.

Proposed Features

Customer and Vendor Features

- Customer registration, login, account, and ticket lookup.
- QR queue join links and OTP-based join verification.
- Live public queue board and ticket cancellation support.
- Vendor onboarding and tenant workspace creation.
- Queue dashboard, walk-in ticket creation, and ticket status controls.
- Locations, store hours, service counters, staff, clients, history, reports, and settings.
- Public board theme customization and asset uploads.

Platform and Billing Features

- Platform admin role and operations portal.
- Overview metrics for tenants, users, revenue, subscriptions, and queue join payments.
- Queue fee configuration by plan.
- Subscription plan management for economical, pro, and enterprise tiers.
- Billing checkout session and queue join payment records.
- PayMongo-oriented payment fields and webhook event records.
- Tenant, user, subscription, payment, and billing-event lists.

Development Platform and Tools

Area	Technology / Tool
Frontend Application	React, Vite, TypeScript, Mantine UI, React Router
Platform Dashboard	Separate React + Vite operations dashboard
Backend	Node.js, Express, TypeScript/JavaScript
Database	PostgreSQL with relational tables, indexes, constraints, and triggers
Real-Time Updates	Server-Sent Events for public queue monitoring
Authentication	JWT-based API authentication, password login, OAuth account support
Notifications	Email/SMS delivery services with delivery logs and local console fallbacks
Payments	PayMongo-ready checkout/session/payment records and webhook route
Abuse Protection	Cloudflare Turnstile-ready public join protection
Asset Storage	Backblaze B2 S3-compatible signed upload URLs for public board assets
Deployment Support	Docker, Docker Compose, environment-based configuration

System Architecture Overview

GetPrio uses a three-application structure connected through a shared backend API and PostgreSQL database. The customer/vendor frontend handles landing pages, authentication, vendor dashboard workflows, public queue joins, and live queue boards. The platform dashboard provides a separate administrative interface for GetPrio operators. The backend exposes REST and streaming endpoints for authentication, queue operations, public access, billing, and platform administration. PostgreSQL stores all core platform records, while external providers are used only when configured.

Data Management

The revised application uses PostgreSQL instead of the earlier MongoDB-oriented proposal. This better supports the updated requirements because the system now has many relational entities: tenants, users, tenant memberships, locations, service counters, staff assignments, tickets, subscriptions, payments, notification deliveries, and billing events.

- Tenant-aware records separate vendor data.
- Daily ticket sequences are scoped by tenant, location, and date.
- Lookup codes support customer ticket access.
- Indexes support queue retrieval, payment monitoring, and history views.

- JSONB fields support flexible theme settings, plan entitlements, and provider metadata.

Methodology

1. **Requirements refinement:** Identify queueing workflows, user roles, tenant boundaries, and platform administration needs.
2. **System design:** Prepare database schema, API route structure, frontend route map, and dashboard layout.
3. **Core implementation:** Build authentication, tenant creation, ticket generation, queue control, and public queue monitoring.
4. **Extended implementation:** Add locations, counters, staff, notifications, public board customization, payments, subscriptions, and platform operations.
5. **Testing and validation:** Test queue ordering, ticket uniqueness, user permissions, payment state records, notification logging, and live queue updates.
6. **Documentation and presentation:** Prepare proposal, system documentation, deployment notes, and defense materials.

Evaluation Criteria

- Functional completeness of customer, vendor, and platform workflows.
- Queue correctness for ticket numbers, positions, and status changes.
- Usability for queue joining, monitoring, and dashboard operation.
- Reliability of API routes, database constraints, and live updates.
- Security and role-appropriate access control.
- Data integrity across tenants, tickets, counters, payments, and subscriptions.
- Maintainability of frontend, backend, platform dashboard, shared types, and database modules.
- Deployment readiness through documented npm and Docker workflows.

Project Timeline

Phase	Activities	Expected Output
Phase 1	Proposal revision, requirements review, scope confirmation	Approved revised proposal
Phase 2	Database and API stabilization	Stable schema, routes, and shared types
Phase 3	Customer and vendor workflow completion	Queue join, public board, dashboard operations
Phase 4	Platform operations and billing workflows	Admin dashboard, plans, fees, payments, subscriptions

Phase 5	Testing, documentation, and deployment preparation	Tested prototype and deployment notes
Phase 6	Capstone presentation preparation	Final paper, presentation, and demonstration script

Expected Outputs

1. A working GetPrio customer/vendor web application.
2. A working GetPrio platform operations dashboard.
3. A backend API with authentication, queue, vendor, public, billing, and platform routes.
4. A PostgreSQL schema and migration set.
5. Local development and deployment documentation.
6. Revised capstone proposal and presentation materials.
7. Demonstration data or workflow scripts for capstone presentation.

Significance of the Study

For customers, GetPrio reduces uncertainty by allowing them to join a queue, receive a ticket, monitor progress, and receive alerts without remaining physically near the service counter. For vendors, it improves queue handling by providing a dashboard for issuing tickets, managing queue movement, organizing staff and counters, and reviewing service history. For platform operators, it demonstrates how a queue management application can be operated as a multi-tenant platform with subscription plans, fee settings, payment monitoring, and administrative oversight.

Conclusion

GetPrio is a revised and expanded capstone project that moves beyond a simple digital queue prototype. The updated application now represents a practical multi-tenant queue management and operations platform. By combining QR-based queue joining, live public monitoring, vendor queue controls, staff and counter management, notification support, payment-aware workflows, subscription plans, public board customization, and platform administration, the project provides a stronger technical and operational foundation for capstone evaluation.

The proposal remains feasible for a web applications development capstone because the workflows are clear, demonstrable, and measurable. At the same time, the revised scope reflects a more complete software product direction that can support real service environments in the future.

End of revised capstone project proposal, Rev. 2